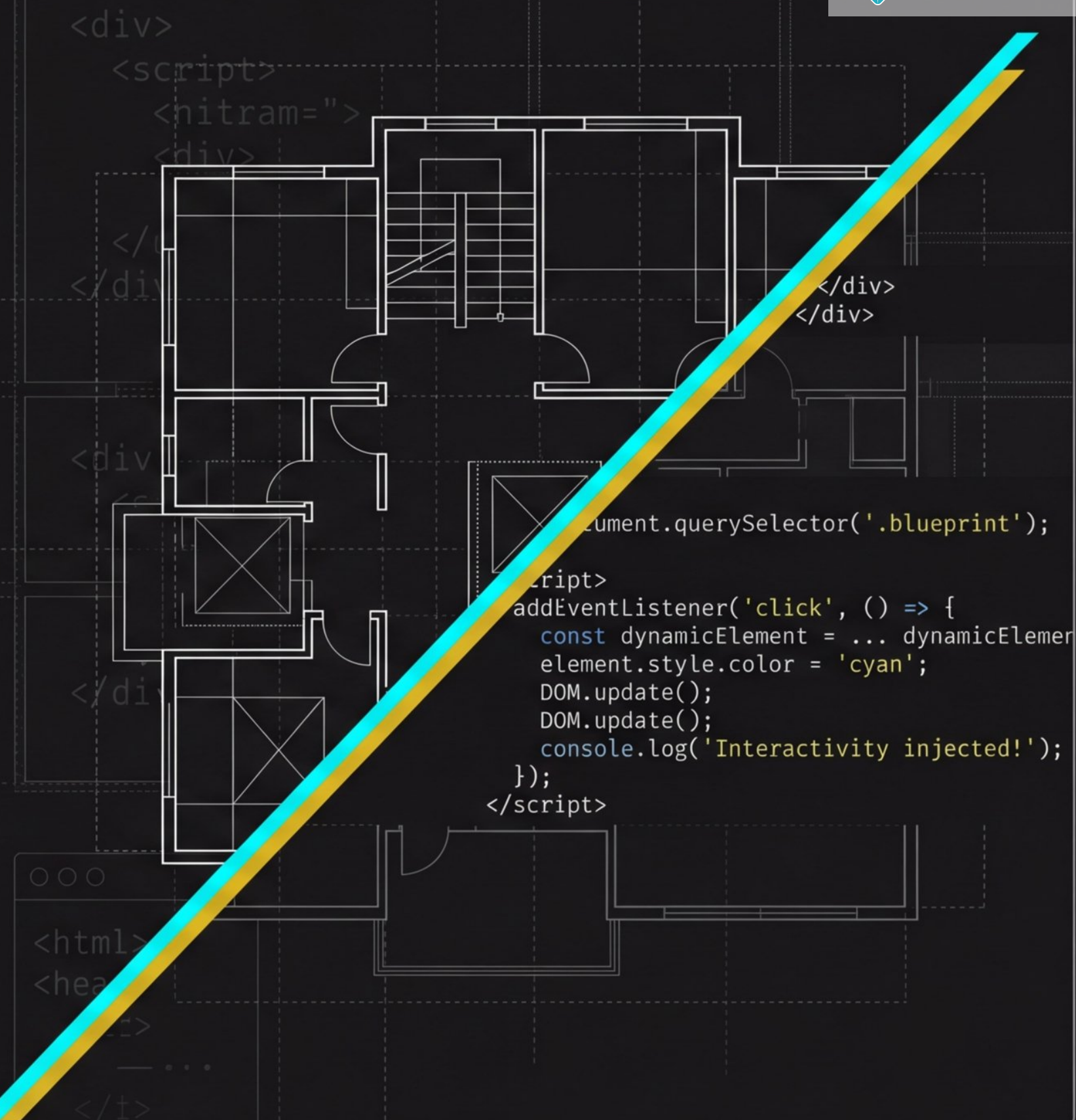


Semana 6: A Ponte entre a Lógica e a Interface

Manipulação do DOM e Interatividade no Lado Cliente

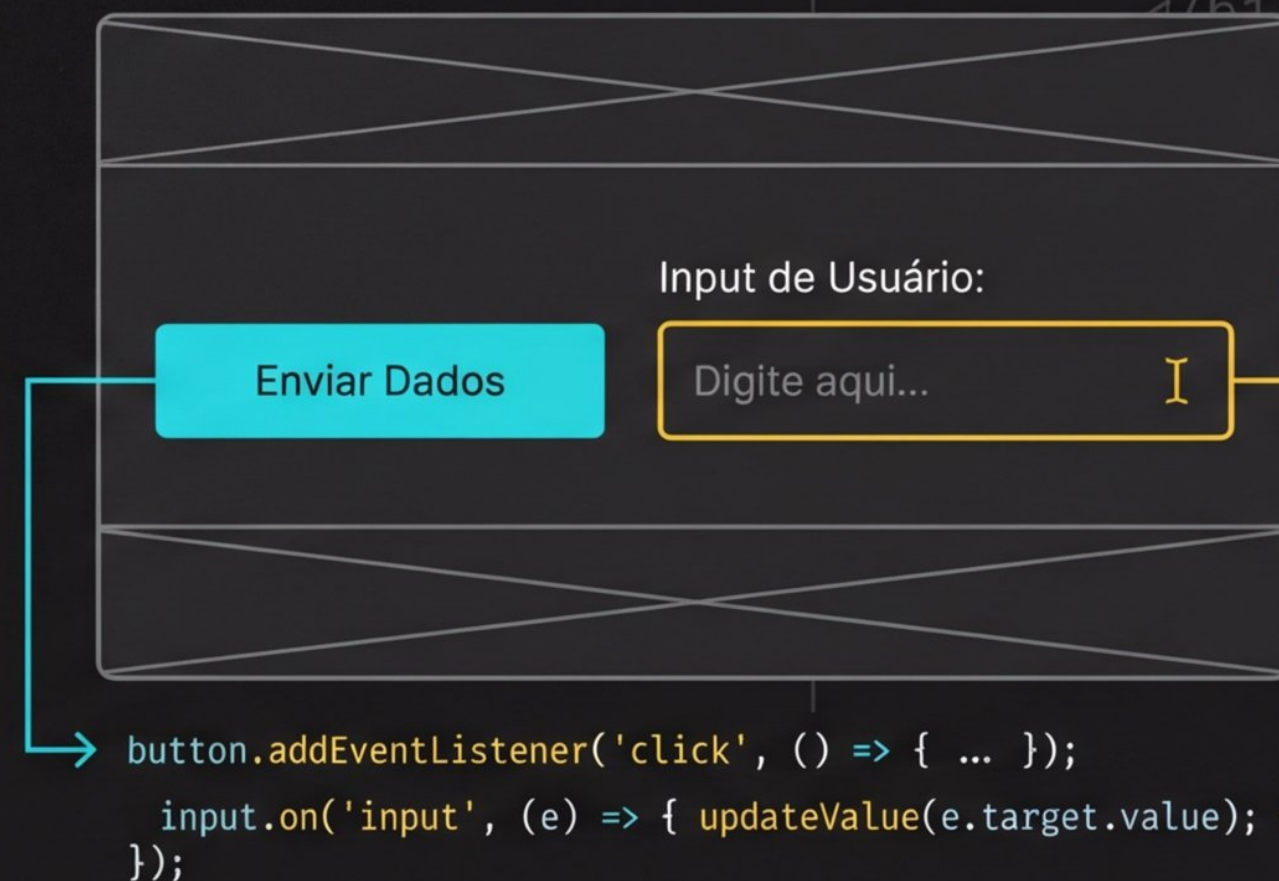


O Código Ganha Vida Visual

Semanas 1-5: A Lógica Invisível

```
let users = [];  
  
const data_processor = (input) => {  
  return input.split(' ');  
};  
  
const config = {  
  debug: false, version: '1.2.0'  
};  
  
for(let i=0; i < 50; i++) {  
  users.push({id: i, status: 'inactive'});  
}
```

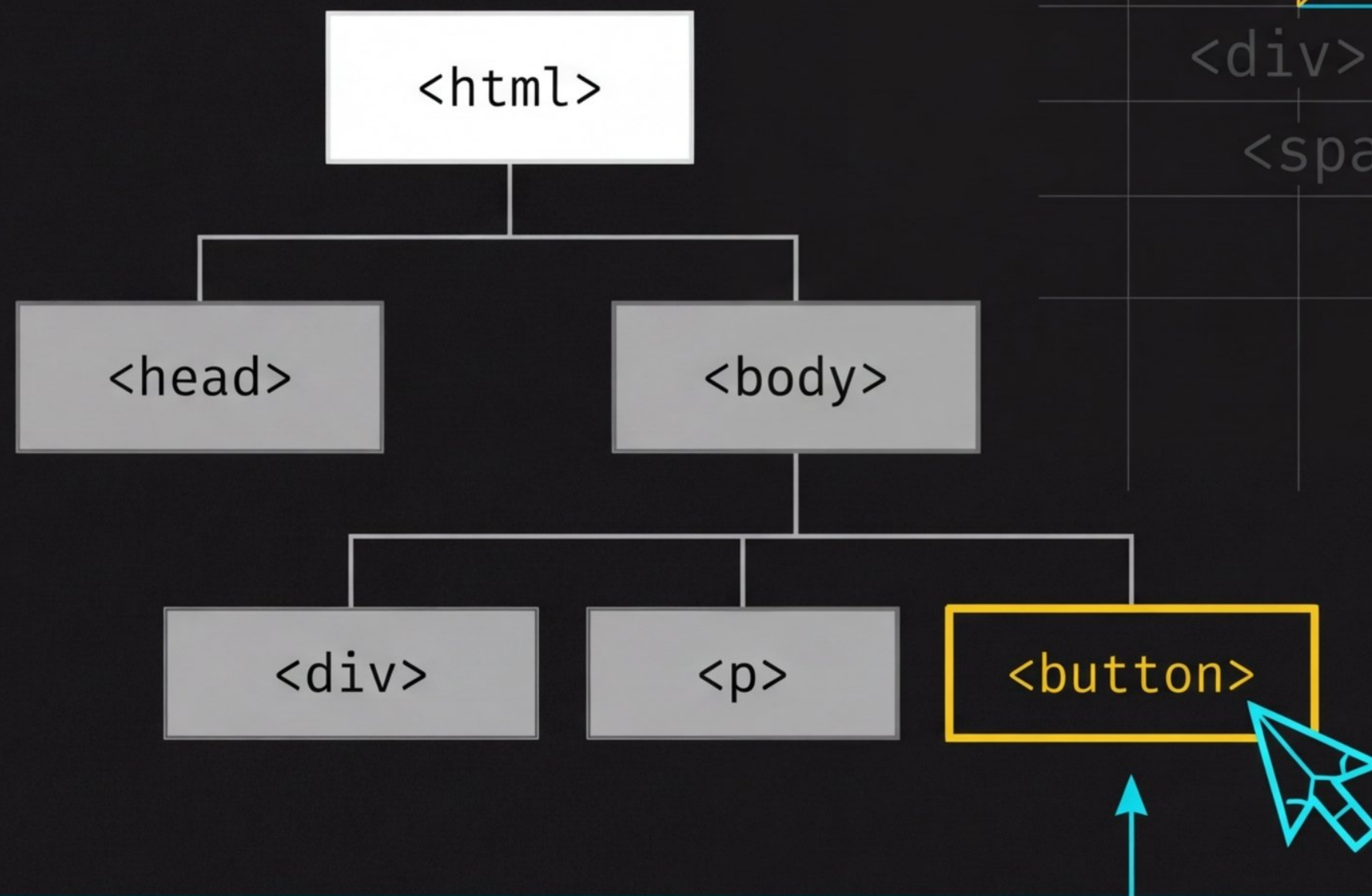
Semana 6: A Interface Reativa



- ✓ 1. Manipular a estrutura visual da página (O **DOM**).
- ✓ 2. Responder instantaneamente a **eventos** do usuário.
- ✓ 3. Criar interatividade contínua sem **recarregar a aba**.

O que é o DOM? (Document Object Model)

O navegador converte as tags HTML estáticas em uma árvore viva de objetos (nós).



O JavaScript usa o DOM como uma ponte para ler, alterar ou destruir esses nós em tempo real.

Os Três Pilares da Interatividade



1. Selecionar

Encontrar o elemento exato na página HTML.



2. Ouvir

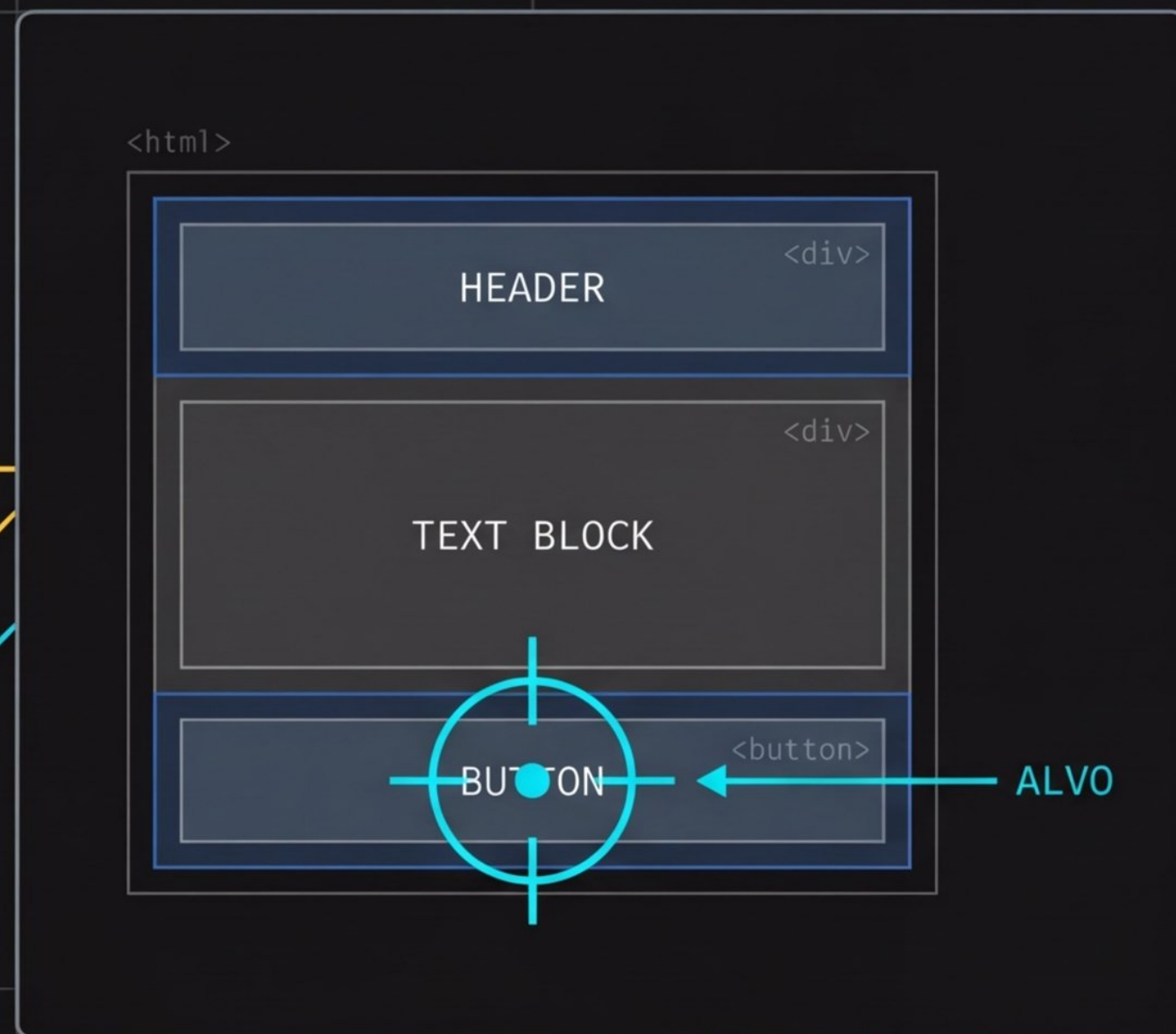
Aguardar a ação ou evento gerado pelo usuário.



3. Manipular

Alterar textos, extrair valores ou mudar estilos visualmente.

Pilar 1: Selecionando Elementos ("Pescando" no DOM)



Captura por ID Único

```
1  
2 document.getElementById("meu-botao")  
...
```

Captura por Regra CSS

```
1  
2 document.querySelector(".minha-classe")  
...
```

Matriz de Decisão: Qual Seletor Usar?

		getElementById	querySelector
Comportamento		Rápido, estrito a IDs únicos	Altamente versátil, aceita qualquer regra CSS
O que retorna?		Apenas um único elemento	O primeiro elemento compatível que encontrar
Exemplo de Sintaxe		("resultado")	("card > p")
Recomendação		Usar quando o elemento tiver um ID claro.	Usar para buscas complexas ou classes.

Pilar 2: Manipulando Conteúdos (Raio-X do DOM)

`.value`

Extraí dados de formulários.

`.innerText`

Lê ou altera texto seguro.

`.innerHTML`

Injeta novas marcações HTML.

`<input>`

`type="text"`

Ana Silva

`<p>`

Bem-vinda de volta.

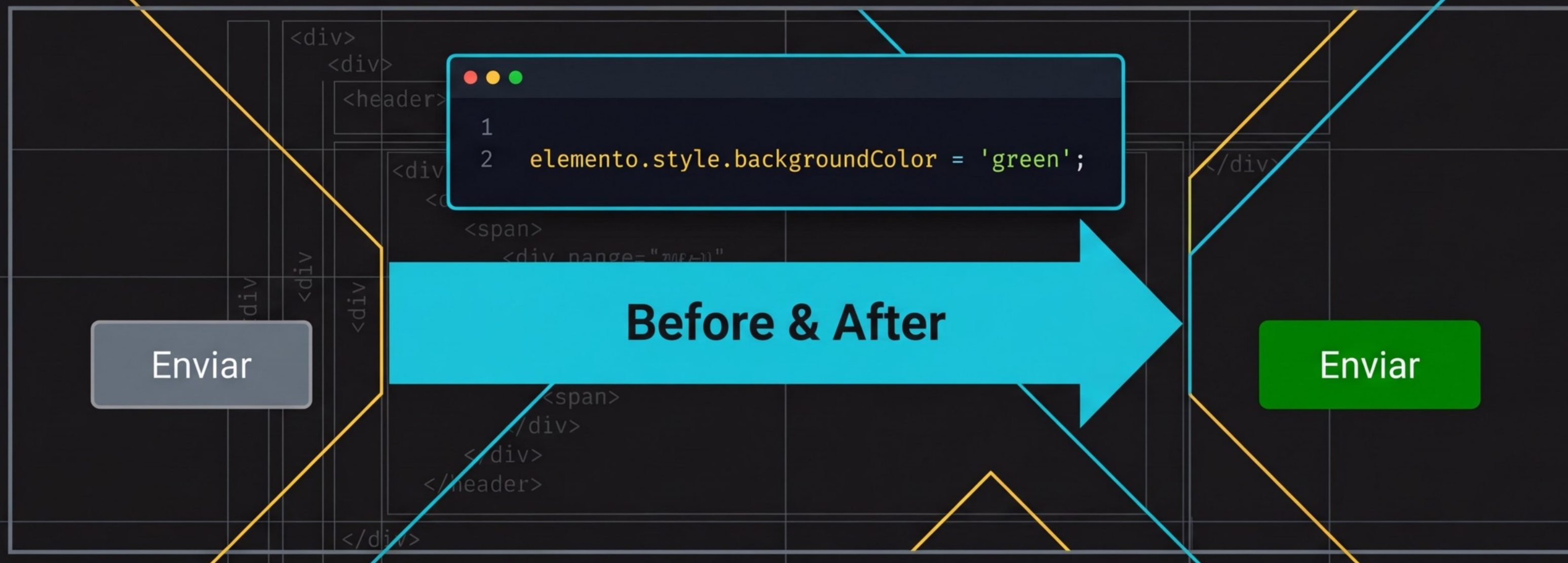
`<div>`

`<strong>Olá, mundo!</strong>`

```
<html>
  <body>
    <div>
      <input type="text" value="Ana Silva" />
      <p>Bem-vinda de volta.</p>
      <div>
        <strong>Olá, mundo!</strong>
      </div>
    </div>
  </body>
</html>
```

Pilar 2: Manipulando Estilos ('CSS Inline Dinâmico')

O JavaScript acessa e altera propriedades CSS em tempo real.



Atenção: Nomes de propriedades CSS com hífen (`background-color`) viram `camelCase` no JavaScript.

<html>

Pilar 3: Eventos e Reações (A Página que "Escuta")

<body>

<div class="container">



<section>

<section>



<click>

Acionado ao clicar com o mouse em botões ou links.



<input>

Acionado instantaneamente a cada tecla digitada em um campo de texto.



<change>

Acionado ao confirmar a alteração de seleções, como em menus dropdown.

Matriz de Boas Práticas: Associando Eventos

Prática Profissional

Evitar: Atributo HTML

```
<button onclick="funcao()">
```

- Mistura responsabilidades.
- HTML e JS ficam emaranhados.
- Difícil manutenção em projetos reais.

Recomendado: Método JS

```
elemento.addEventListener('click',  
funcao)
```

- Separação estrita de responsabilidades.
- O HTML fica limpo e legível.
- Permite associar múltiplos eventos ao mesmo elemento.

Síntese: O Ciclo Contínuo da Interatividade Web



Situação-Problema: A Aplicação Real (ABP)

O Cenário

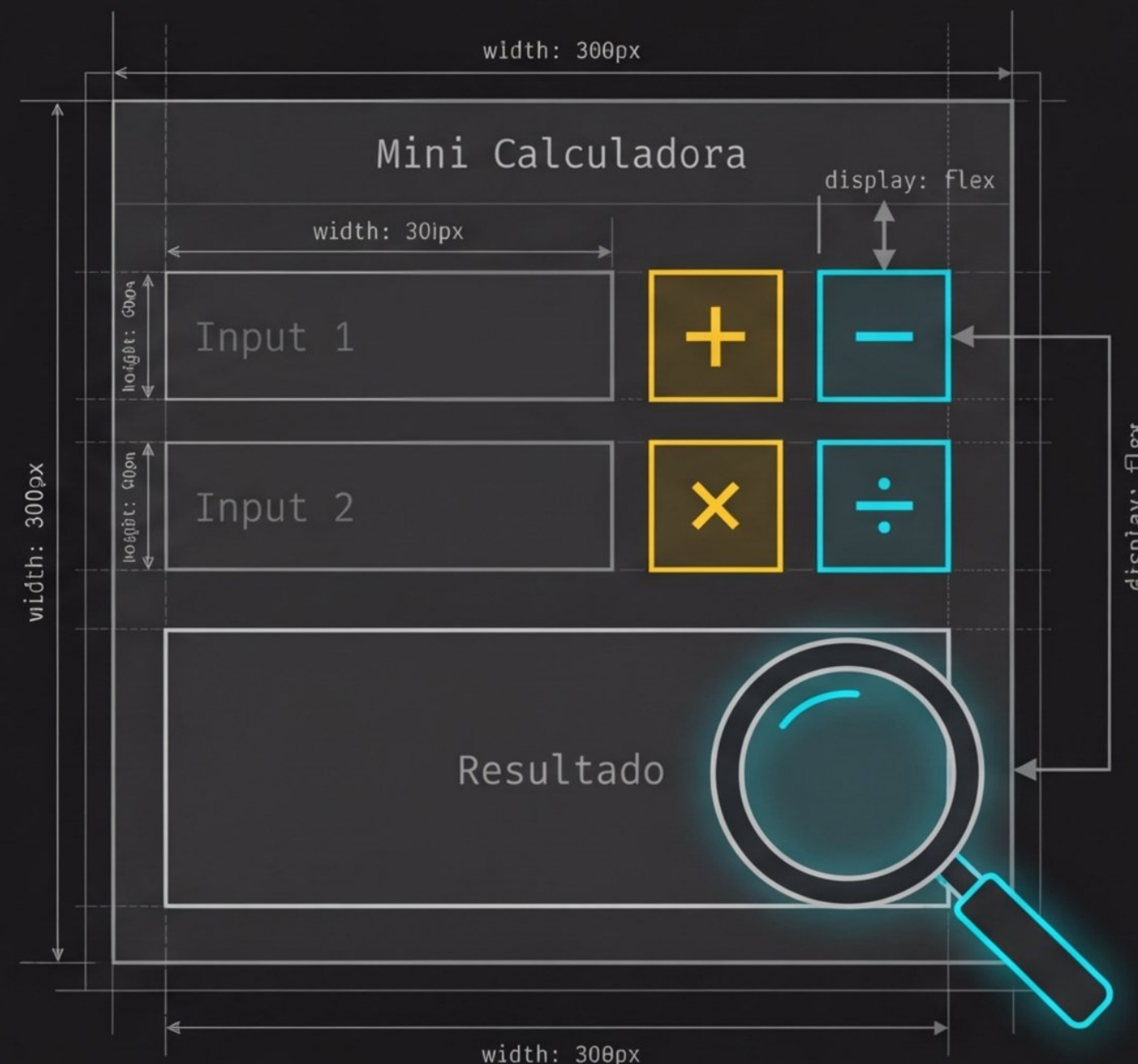
Sistemas modernos exigem atualizações instantâneas sem recarregar a página.

O Desafio

Desenvolver um 'Contador de Cliques' ou 'Mini Calculadora' onde os cálculos reflitam na tela no exato milissegundo em que ocorrem.

O Objetivo

Aplicar Seleção (**DOM**), Escuta (**Eventos**) e Manipulação (**Lógica**) simultaneamente em uma única interface.

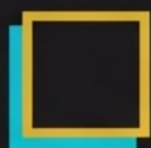


Roteiro da Prática (Laboratório)



Estruturar

Criar campos (`<input>`) e botões (`<button>`) estáticos no HTML.



Mapear

Selecionar os elementos no arquivo JS usando `querySelector`.



Escutar

Adicionar `addEventListener` aos botões para capturar os cliques.



Reagir

Extrair os dados digitados, processar a lógica e exibir o resultado alterando o `.innerText`.

Atividade Verificadora da Semana

A Missão

Criar uma página interativa com um formulário. Ao clicar em "Resumir", o sistema deve capturar os dados digitados e gerar um "Card de Resumo" na própria página, sem recarregamento.

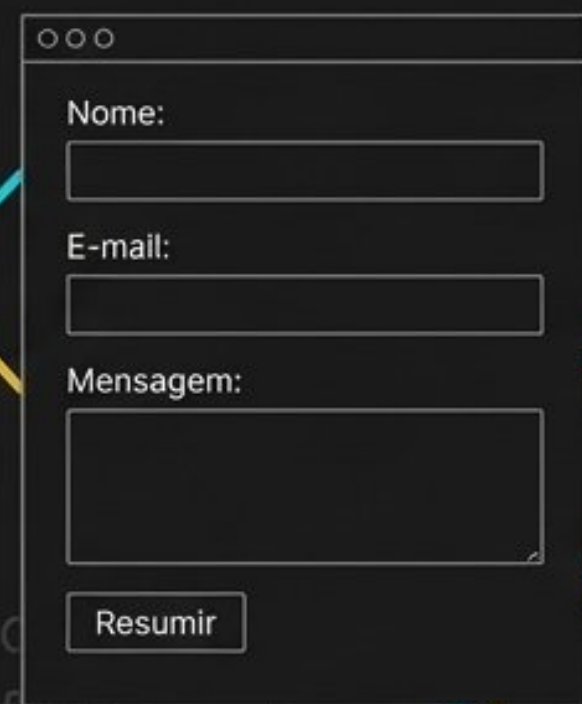


Diagram illustrating the initial form structure:

```
<div>  
<form>  
  Nome:   
  E-mail:   
  Mensagem:   
  Resumir  
</form>  
</div>
```



Diagram illustrating the generated "Card de Resumo" structure:

```
<div>  
  Card de Resumo  
  Nome: [Valor]  
  E-mail: [Valor]  
  Mensagem: [Valor]  
</div>
```

Critérios de Entrega



Correção na seleção de elementos (Sem **IDs** quebrados).



Uso estrito de **addEventListener** para capturar a ação.



Atualização limpa do **DOM** usando **.innerText** ou **.innerHTML**.

Conclusão: O Domínio da Interface

Seu código JavaScript agora afeta o mundo real visual. O terminal invisível tornou-se uma interface responsiva.

Próximos Passos (Semana 7)

Validação de Formulários **Client-Side**. Usaremos essas mesmas técnicas do **DOM** para blindar aplicações contra dados incorretos antes mesmo de chegarem ao servidor.